Laboratorios Systemd Red Hat 7

Systemd es un administrador del sistema y servicios de GNU/Linux. Se ha diseñado para mantener la compatibilidad con los scripts SysV. Systemd sustituye a Upstart en Red Hat 7.

Systemd introduce el concepto de **`unidades systemd`**. Estas unidades están configuradas en archivos localizados en las siguientes ubicaciones:

- **/usr/lib/systemd/system/**: unidades distribuidas con paquetes RPM instalados.
- **/run/systemd/system/:** unidades creadas en tiempo de ejecución. Tiene precedencia sobre el directorio anterior.
- **/etc/systemd/system/:** unidades creadas y administradas por el administrador del sistema. Este directorio tiene precedencia sobre el directorio anterior.

Comprobar si systemd está funcionando:

Systemd se está ejecutando como daemon padre (PID = 1)

ps -eaf | grep systemd

root 1 0 0 10:41 ? 00:00:01 /usr/lib/systemd/systemd --switched-root --system --deserialize 21

root 610 1 0 10:41 ? 00:00:00 /usr/lib/systemd/systemd-journald

root 650 1 0 10:41 ? 00:00:00 /usr/lib/systemd/systemd-udevd

root 776 1 0 10:41 ? 00:00:00 /usr/lib/systemd/systemd-logind

dbus 794 1 0 10:41 ? 00:00:00 /bin/dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation

systemd+ 1137 1 0 10:41 ? 00:00:00 /usr/lib/systemd/systemd-resolved

root 3307 2674 0 11:34 pts/0 00:00:00 grep --color=auto systemd

Comprobando versión:

 # systemctl --version

Systemd proporciona diferentes tipos de unidades. Dichos tipos de unidad se distinguen según la extensión de archivo correspondiente.

systemctl list-units --all

El comando anterior mostrará cualquier unidad cargada o que se haya intentado cargar, independientemente de su estado en el sistema. Algunas unidades pasan a estar inactivas después haberse ejecutado, mientras que otras pueden no haber sido encontradas en disco.

Se pueden utilizar otras opciones en el comando para filtrar las unidades mostradas, como por ejemplo **--state=** para indicar el estado o **--type** para indicar el tipo de unidad.

El comando **list-units** solo muestra las unidades que `systemd` ha intentado cargar (directa o indirectamente a través de dependencias). Systemd solo intenta cargar las unidades que necesita en su árbol de dependencias, pero existen otras unidades. Para conocerlas, ejecutamos:

systemctl list-unit-files

El estado que se muestra con este comando puede ser **enabled, disabled, static**, o **masked**. En este contexto, **static** significa que la unidad no contiene una sección **install**, lo que quiere decir que no está destinada a ser ejecutada directamente (puede serlo via dependencia). Las unidades **masked** han sido administrativamente bloqueadas, de forma que no pueden ser iniciadas. El resto de estados lo iremos viendo a continuación

systemctl show nombre_unidad

El comando anterior permite mostrar los detalles de una cierta unidad.

Laboratorio 1. De entre todas las unidades que hay en el sistema, muestra los detalles de una unidad de tu elección.

Administrando servicios del sistema

Upstart (la versión anterior de Red Hat) utiliza para gestionar servicios los scripts ubicados en `/etc/rc.d/init.d/`. Estos scripts están escritos en Bash, y permiten tener control sobre el estado de los servicios y demonios en sus sistemas. En Red Hat 7, estos scripts han sido reemplazados con `unidades de servicio` (service units)

Las unidades de servicio tienen la extensión `.service` y tienen el mismo propósito que los scritps init. Para iniciar, detener, reiniciar, activar o desactivar un cierto servicio, se utiliza el comando `systemctl`

Los comandos `service` y `chkconfig` siguen siendo válidos, pero solamente están incluidos con propósitos de compatibilidad con sistemas antiguos, y deben ser evitados.

Listado de servicios activos

Para comprobar todos las unidades de servicio cargadas, ejecutamos el comando:

systemctl list-units --type service

Para cada unidad de servicio, este comando muestra su nombre completo (UNIT) seguido de una anotación indicando si la unidad se ha cargado (LOAD), su estado activación (ACTIVE ó SUB) y una breve descripción (DESCRIPTION).

Por defecto, **`systemctl list-units`** muestra solamente las unidades activas. Si deseamos mostrar todas las unidades independientemente de su estado, utilizamos el parámetro **`--all`.**

systemctl list-units --type service --all

También se pueden listar las unidades de servicio disponibles, y comprobar así si están activadas:

systemctl list-unit-files --type service

Para comprobar los detalles sobre una cierta unidad de servicio:

systemctl status nombre-del-servicio.service

La información que proporciona este comando es:

- **Loaded** - Información sobre si la unidad servicio ha sido cargada, la ruta absoluta la unidad de servicio y si la unidad está activa.
- **Active** - Información sobre si la unidad de servicio está ejecutándose, con timestamp incluido.
- **Main PID** - El PID del servicio seguido de su nombre.
- **Status** - Información adicional sobre el servicio.
- **Process** - Información adicional sobre procesos relacionados.
- **CGroup** - Información adicional.

Para comprobar simplemente si una unidad de servicio está ejecutándose:

systemctl is-active nombre-del-servicio.service

Del mismo modo, para comprobar si una unidad de servicio está activada (configurada para iniciar cuando el sistema arranca):

systemctl is-enabled nombre-del-servicio.service

Una unidad de servicio puede tener los siguientes estados:

- **enabled:** configurada para ejecutarse durante el inicio del sistema.
- **disabled:** configurada para no ejecutarse durante el inicio del sistema.
- **active:** unidad que se está ejecutando actualmente.
- **inactive:** unidad que no se está ejecutando actualmente.
- **failed:** unidad que ha fallado durante su inicio.

Una unidad que no está activada puede estar activa si ha sido arrancada durante la sesión. Del mismo modo, una unidad puede estár inactiva, a pesar de estar activada.

Iniciar un servicio

Para iniciar un servicio:

systemctl start nombre-del-servicio.service

Este comando incia el servicio en esta sesión solamente, de modo que en el siguiente reinicio el servicio solo se ejecutará si la unidad de servicio está activa.

Para detener un servicio:

systemctl stop nombre-del-servicio.service

Detener un servicio es diferente a desactivarlo, ya que si simplemente lo detenemos, el servicio volverá a iniciarse automáticamente durante el próximo inicio.

Para reiniciar un servicio:

systemctl restart nombre-del-servicio.service

Si todo va bien, no mostrará nada por pantalla. Este comando solamente tiene sentido en caso de que el servicio correndiente ya está ejecutándose.

Si queremos que un servicio únicamente se reinicie en caso de estar ejecutándose:

systemctl try-restart nombre-del-servicio.service

Algunos servicios permiten que solamente se recargue su configuración sin interrumpir su ejecución. Esto es interesante en el caso de ciertos servicios que no se pueden o deben parar, pero cuya configuración hay que modificar. Para ello:

systemctl reload nombre-del-servicio.service

Activando un servicio

Para configurar una unidad de servicio para que se inicie durante el arranque del sistema:

systemctl enable nombre-del-servicio

Este comando crea un enlace simbólico en **/etc/systemd/system** apuntando a **/usr/lib/systemd/nombre-del-servicio.service**, pero no lo recrea en caso de existir. Si deseamos asegurarnos de que el enlace sea efectivamente creado:

systemctl reenable nombre-del-servicio.service

Con **`reenable`** el enlace simbólico es eliminado y se crea de nuevo.

Desactivando un servicio

Para evitar que un servicio se inicie durante el arranque del sistema:

systemctl disable nombre-del-servicio.service

Este comando borra el enlace simbólico de /etc/systemd/system a la unidad de servicio, ubicada en /usr/lib/systemd/system/nombre-del-servicio.service.

Para que un servicio no pueda ser iniciado manualmente o bien por otro servicio:

systemctl mask nombre-del-servicio.service

Este comando reemplaza el enlace de /etc/systemd/system a la unidad de servicio, por un enlace a /dev/null. Para invertir el proceso:

systemctl unask nombre-del-servicio.service

Laboratorio 2. Comprueba si tienes instalado el servidor web Apache (httpd). En caso contrario, instálalo. Una vez que se instale, comprueba si está activado.

Laboratorio 3. Desactiva el servidor Apache (si no está desactivado ya). Reinicia Red Hat y comprueba si está iniciado.

Laboratorio 4. Con el servicio Apache desactivado, ejecútalo y comprueba que está activo.

Laboratorio 5. Finalmente, activa el servicio Apache. Después reinicia y comprueba que está activo.

Targets

En Red Hat 7, el concepto de runlevel ha sido sustituido por `objetivos` (target). Los target, se representan mediante archivos con extensión .target, y su propósito es agrupar otras unidades de systemd a través de una cadena de dependencias. Por ejemplo, la unidad **`graphical.target`** se usa para iniciar la sesión gráfica, inicia servicios como GNOME Display Manager (gdm.service) o el servicio Accounts (accounts-daemon.service).

Además activa la **unidad** **`multi-user.target`**. Del mismo modo, la unidad `multi-user-target` inicia otros servicios necesarios, como NetworkManager (NetworkManager.service) además de activar otra unidad llamada `basic.target`.

Aunque no hay un orden estricto en que se inician los servicios cuando se arranca un sistema Red Hat 7 (con systemd), hay una estructura para el proceso de arranque. Todo depende del archivo **default.target** ubicado en /**etc/systemd/system**.

Este archivo, es en realidad un enlace simbólico a un target ubicado en **`/usr/lib/systemd/system`**, donde se muestra qué targets se iniciarán durante el arranque. Se puede ver el contenido de dicho target en **/user/lib/systemd/system/default.target**.

Para comprobar el target por defecto:

systemctl get-default

El archivo **default.target** contiene entre otros los siguientes parámetros:

- **`Requires:`** iniciar la unidad citada. Si el target citado está desactivado, el target también se desactivará
- **`Wants:`** iniciar la unidad citada de una forma menos exigente que "Requires". Si las unidades citadas no se inician, no tiene impacto sobre la operación en su conjunto. Las dependencias de este tipo también se pueden configurar fuera del archivo de configuración de la unidad, mediante la adición de enlaces simbólicos a un directorio .wants que acompaña a la unidad.
- **`Before:`** unidades que se cargarán después que el target.
- **`After:`** unidades que se cargarán antes que el target. Suele incluirse junto con la opción Requires. Si dos unidades no tienen dependencias de orden, se cargarán simultáneamente.
- **`Conflicts:`** detiene la unidad o unidades citadas ya que son incompatibles.

Tabla de targets

Runlevel de SysV	Target de systemd	Notas
0	runlevel0.target, poweroff.target	Detiene el sistema.
1, s, single	runlevel1.target, rescue.target	Modalidad de usuario único.
2, 4	runlevel2.target, runlevel4.target, multi-user.target	Definidos por el usuario. Preconfigurados a 3.
3	runlevel3.target, multi-user.target	Multiusuario, no gráfica. Los usuarios, por lo general, pueden acceder a través de múltiples consolas o a través de la red.
5	runlevel5.target, graphical.target	Multiusuario, gráfica. Por lo general, tiene todos los servicios del nivel de ejecución 3, además de un inicio de sesión gráfica.
6	runlevel6.target, reboot.target	Reinicia el sistema.
emergency	emergency.target	Consola de emergencia.

Para comprobar los targets que han sido cargados:

systemctl list-units --type target

Si lo que queremos es mostrar todas las unidades cargadas, independientemente de su estado, añadimos la opción --all al final:

systemctl list-units --type target --all

Cambiar el target

Para cambiar el target por defecto:

systemctl set-default nombre-del-target.target

Para cambiar a un target diferente durante la sesión actual:

systemctl isolate nombre-del-target.target

Laboratorio 6. Cambia al target multiuser. Después vuelve al target por defecto.

Laboratorio 7. Redefine el target multiuser como el target por defecto. Después reinicia y comprueba que no se arranca el entorno gráfico.

Cambiar al modo de recuperación

Podemos desear cambiar al modo de recuperación una vez que hemos iniciado el sistema. Esto puede ser interesante si queremos poner el sistema en un estado básico, desactivando servicios que no son imprescindibles (como red), para iniciar tareas de recuperación.

systemctl rescue

Cambiar al modo de emergencia

El modo de emergencia provee el entorno mínimo para recuperar el sistema en situaciones en que no se puede acceder al modo de recuperación. En este modo el sistema de archivos se monta el punto de montaje raíz en modo de solo lectura, no se montan otros puntos de montaje ni el servicio de red, y solamente unos cuantos servicios son iniciados. Para cambiar al modo de emergencia:

systemctl emergency

Cambiar el target vigente

En *systemd* los targets quedan expuestos a través de «target units». Se pueden cambiar de esta manera:

 # systemctl isolate multi-user.target

Esto solo cambiará el target actual, y no tendrá ningún efecto sobre el siguiente arranque. Esto es equivalente a las órdenes `telinit 3` o `telinit 5` en Sysvinit.

Gestión de apagado/encendido

Algunos comandos útiles son:

- **systemctl halt**: Apaga abruptamente el sistema
- **systemctl poweroff**: Apaga el sistema
- **systemctl reboot**: Reinicia el sistema
- **systemctl suspend**: Suspende el sistema
- **systemctl hibernate**: Hiberna el sistema
- **systemctl hybrid-sleep**: Hiberna y suspende el sistema

Controlar Systemd en una máquina remota

Para controlar Systemd en otra máquina:

systemctl --host nombre_usuario@nombre_maquina comando

O bien, también se puede utilizar SSH:

Puede utilizar las siguientes órdenes `systemctl` con el parámetro **`-H`** *`usario@host`* para controlar una instancia de systemd en una máquina remota. Esto utilizará SSH para conectarse a la instancia systemd remota.

Escribir archivos .service personalizados

La sintaxis de los archivos de unidad de *systemd* se inspira en los archivos .desktop de XDG Desktop Entry Specification, que, a su vez, están inspirados en los archivos .ini de Microsoft Windows.

Manejar las dependencias

Con *systemd* las dependencias pueden ser resueltas planificando la unidad correctamente.

El caso más típico es que la unidad `A` requiere la unidad `B` para poder funcionar, por lo que esta última debe iniciarse antes que `A`.

En ese caso, agregue `Requires=B` y `After=B` a la sección `[Unit]` de `A`. Si la dependencia es opcional agregue, en su lugar, `Wants=B` y `After=B`. Tenga en cuenta que `Wants=` y `Requires=` no incluyen `After=`, lo que significa que si `After=` no esté especificado, las dos unidades se iniciarán en paralelo.

Las dependencias se colocan normalmente en los archivos .service y no en los .target. Por ejemplo, `network.target` es llamado por cualquiera que sea el servicio que configure las interfaces de red, por lo tanto, la solicitud que hace después la propia unidad personalizada es suficiente, ya que `network.target` se inicia de todos modos.

Type

Existen diferentes tipos de arranque a tener en cuenta cuando se escribe un archivo de servicio personalizado. Esto se configura mediante el parámetro `Type=` en la sección `[Service]`. Consulte `man systemd.service` para una explicación más detallada.

- **`Type=simple:`** *systemd* considera que el servicio debe iniciarse inmediatamente. El proceso no debe romperse. No utilice este tipo si otros servicios tienen que ser llamados por ese servicio, a menos que no sea activado por el socket.
- **`Type=forking:`** *systemd* considera que el servicio debe ser iniciado antes que el proceso se rompa y el antecesor se haya terminado. Para los demonios clásicos use este tipo a menos que sepa que no es necesario, ya que la mayoría de los demonios usan doble bifurcación para indicar que están listos. Debe especificar también `PIDFile=` para que *systemd* puede realizar un seguimiento del proceso principal.
- **`Type=oneshot:`** Esto es útil para los scripts que hacen un solo trabajo y luego concluyen. Es posible que desee también establecer `RemainAfterExit=yes` de modo que *systemd* sigue considerando el servicio como activo después de que el proceso haya terminado.
- **`Type=notify:`** Igual que `Type=simple`, pero con la condición de que el demonio va a enviar una señal a *systemd* cuando esté listo. Esto requiere del código específico proporcionado por `libsystemd-daemon.so`.
- **`Type=dbus:`** El servicio se considera listo cuando el `BusName` especificado aparece en el bus del sistema DBus.

Modificar los archivos de unidad suministrados

Para editar un archivo de unidad proporcionado por un paquete, podemos crear un directorio llamado /**etc/systemd/system/*unit*.d/** por ejemplo /etc/systemd/system/httpd.service.d/ y colocar los archivos ***.conf** en dicho directorio para reemplazarlos o añadir nuevas opciones.

systemd analizará estos archivos *.conf y los aplicará antes que los de la unidad original.

Por ejemplo, si deseamos simplemente agregar una dependencia adicional a una unidad, podemos crear el siguiente archivo:

/etc/systemd/system/*unit*.d/customdependency.conf

[Unit]

Requires=*dependencia nueva*

After=*dependencia nueva*

Siguiendo otro ejemplo, con el fin de reemplazar la directiva ExecStart para una unidad que no es del tipo oneshot, crearemos el siguiente archivo:

/etc/systemd/system/*unit*.d/customexec.conf
[Service]

ExecStart=

ExecStart=*orden nueva*

Otro último ejemplo, para reiniciar automáticamente un servicio:

/etc/systemd/system/*unit*.d/restart.conf

[Service]

Restart=always

RestartSec=30

A continuación, ejecutaremos lo que sigue para que los cambios surtan efecto:

systemctl daemon-reload
systemctl restart *unidad*

Por otro lado, podemos copiar el archivo de la antigua unidad desde /usr/lib/systemd/system/ a /etc/systemd/system/ y realizar los cambios allí. Un archivo de unidad **ubicado en /etc/systemd/system**/ siempre tiene preferencia sobre la misma unidad localizada en /usr/lib/systemd/system/.

Debemos tener en cuenta que cuando la unidad original localizada en /usr/lib/ ha cambiado debido a una actualización del paquete que lo suministra, estos cambios no se aplicarán automáticamente al archivo de unidad personalizada ubicado en /etc/.

De este modo, tendremos que volver a activar manualmente la unidad con la orden systemctl reenable *unidad*. Por consiguiente, se recomienda utilizar el método *.conf descrito anteriormente.

Sugerencia:

Podemos utilizar la orden **systemd-delta** para ver qué archivos de la unidad han sido invalidados y cuáles han cambiado.

Como los archivos de unidad suministrados se actualizarán de vez en cuando, es conveniente utilizar **systemd-delta** para tareas de mantenimiento del sistema.

Crear un script que se ejecute durante el proceso de arranque

Supongamos que contamos con un script llamado `mi-script.sh`, que queremos que se ejecute durante el arranque. Entonces es preciso realizar el siguiente procedimiento:

1. Crear un nuevo archivo llamado `mi_script.service` /etc/systemd/system.

   NOTA: El script `mi-script.sh` debe tener permisos de ejecución.

2. Comprobar el target por defecto:

   systemctl get-default

3. Supongamos que el target por defecto es `graphical.target`. Entonces deberemos añadir el siguiente contenido al archivo `mi_script.service`:

   [Unit]
   Description=My script
   [Service]
   ExecStart=/usr/bin/mi-script.sh
   [Install]
   WantedBy=multi-user.target

4. Finalmente, debemos habilitar el servicio `mi_script.service`:

   systemctl enable mi_script.service

Laboratorio 8. Crea un script llamado iniciar-supervisamem que llama a un archivo llamado supervisamem en el directorio /opt/scripts/. Después configura systemd para que lo ejecute durante el inicio del sistema.

En este laboratorio, agregaremos la aplicación supervisamem, como servicio a través de systemd.

Usted desea disponer en un nivel de ejecución determinado, de registros periódicos del consumo de memoria del servidor. Para ello creara el programa encargado de realizar dicho registro periódicamente, así como un script de inicio normalizado y que pueda utilizar **systemd**.

Cree en el directorio **/opt/scripts** con su editor favorito el archivo **supervisamem** con el siguiente contenido:

```
#!/bin/bash
while true
do
ahora=$(date "+%H:%M:%S - ")
echo -n $ahora >> /var/log/supervisamem.log
grep Dirty /proc/meminfo >> /var/log/supervisamem.log
sleep 30
done
```

Muestra la página del manual asociada con una unidad (esto tiene que ser apoyado por el archivo .unit):

 # systemctl help *unidad*
systemctl help NetworkManager.service

Recarga *systemd*, escaneando en busca de unidades nuevas o modificadas:

 # systemctl daemon-reload

Operaciones:

1. Hacer este archivo ejecutable

2. Crear en el directorio /etc/init.d un script normalizado de inicio de servicio para la aplicacion supervisamem

3. Hacer este archivo ejecutable

4. Comprobar el buen funcionamiento del programa ejecutando el servicio correspondiente.

5. Dejar que funcione unos minutos

6. Comprobar el contenido del archivo /var/log/supervisamem.log

 # cd /etc/systemd/system

#vi supervisamem.service

[Unit]

Description=Daemon of axample

[Service]

ExecStart=/opt/scripts/iniciar-supervisamem start

ExecStop=/opt/scripts/iniciar-supervisamem stop

Type=forking

[Install]

WantedBy=multi-user.target

Tenemos que refrescar al systemd para que lea de nuevo las configuraciones, esto lo hacemos con systemctl daemon-reload

#systemctl daemon-reload

 # cd /opt/scripts/

 # ls

supervisamem iniciar-supervisamem

systemctl start supervisamem

systemctl stop supervisamem

Comprobaremos con el comando ps, y con el comando tail, si el servicio se inicia y se detiene correctamente.

 # tail -f /var/log/supervisamem.log

 # ps -ef

Type

Existen diferentes tipos de arranque a tener en cuenta cuando se escribe un archivo de servicio personalizado. Esto se configura mediante el parámetro `Type=` en la sección `[Service]`. Consulte `man systemd.service` para una explicación más detallada.

- **`Type=simple:`** *systemd* considera que el servicio debe iniciarse inmediatamente. El proceso no debe romperse. No utilice este tipo si otros servicios tienen que ser llamados por ese servicio, a menos que no sea activado por el socket.
- **`Type=forking:`** *systemd* considera que el servicio debe ser iniciado antes que el proceso se rompa y el antecesor se haya terminado. Para los demonios clásicos use este tipo a menos que sepa que no es necesario, ya que la mayoría de los demonios usan doble bifurcación para indicar que están listos. Debe especificar también `PIDFile=` para que *systemd* puede realizar un seguimiento del proceso principal.
- **`Type=oneshot:`** Esto es útil para los scripts que hacen un solo trabajo y luego concluyen. Es posible que desee también establecer `RemainAfterExit=yes` de modo que *systemd* sigue considerando el servicio como activo después de que el proceso haya terminado.
- **`Type=notify:`** Igual que `Type=simple`, pero con la condición de que el demonio va a enviar una señal a *systemd* cuando esté listo. Esto requiere del código específico proporcionado por `libsystemd-daemon.so`.
- **`Type=dbus`**: El servicio se considera listo cuando el `BusName` especificado aparece en el bus del sistema DBus.

Importante:

Si los servicios no tienen una sección **[Install]** significa, por lo general, que se les llama de forma automática por otros servicios. Pero si necesita instalarlos manualmente, utilice la orden siguiente, reemplazando foo con el nombre del servicio.

ln -s /usr/lib/systemd/system/*foo*.service /etc/systemd/system/graphical.target.wants/

Mounts

Tradicionalmente en GNU/Linux, el montaje automático de unidades se venía haciendo mediante el archivo **`/etc/fstab`**. Pero esto ha cambiado con la llegada de Systemd. Systemd maneja el automontaje de las unidades.

El archivo `/etc/fstab` es utilizado por Systemd para generar unidades mount. En primer lugar vamos a ver cómo funciona fstab.

fstab

El archivo **/etc/fstab** es usado para definir cómo las particiones, distintos dispositivos de bloques o sistemas de archivos remotos deben ser montados e integrados en el sistema. El posible contenido de un archivo fstab podría ser:

```
#
UUID=24f28fc6-717e-4bcd-a5f7-32b959024e26 / ext4 defaults,noatime 0 1
UUID=03ec5dd3-45c0-4f95-a363-61ff321a09ff /home ext4 defaults,noatime 0 2
UUID=4209c845-f495-4c43-8a03-5363dd433153 none swap defaults 0 0
```

Cada línea incluye la siguiente información:

- **UUID -** Define el dispositivo que va a ser montado. Es preferible usar la UUID a "/dev/x" puesto que la UUID es independiente del nombre que el sistema otorga al dispositivo.
- *directorio* - Indica el punto de montaje de la partición.
- tipo de partición - indica el sistema de archivos de la partición. Algunos ejemplos pueden ser ext2, ext3, ext4, xfs, vfat o auto. el valor `auto` permite al comando `mount` (que es en última instancia quien monta el dispositivo) determine el tipo de partición.
- **opciones** - Indica las opciones de montaje que `mount` empleará. Algunas opciones son:
  - **auto** - El sistema de archivos será montado automáticamente durante el arranque, o cuando la orden mount -a se invoque.
  - **noauto** - El sistema de archivos no será montado automáticamente, solo cuando se le ordene manualmente.
  - **exec** - Permite la ejecución de binarios residentes en el sistema de archivos.
  - **noexec** - No permite la ejecución de binarios que se encuentren en el sistema de archivos.
  - **ro** - Monta el sistema de archivos en modo sólo lectura.
  - **rw** - Monta el sistema de archivos en modo lectura-escritura.
  - **user** - Permite a cualquier usuario montar el sistema de archivos. Esta opción incluye **noexec, nosuid, nodev**, a menos que se indique lo contrario.
  - **users** - Permite que cualquier usuario perteneciente al grupo users montar el sistema de archivos.
  - **nouser** - Solo el usuario root puede montar el sistema de archivos.
  - **owner** - Permite al propietario del dispositivo montarlo.
  - **sync** - Todo el I/O se debe hacer de forma sincrónica.
  - **async** - Todo el I/O se debe hacer de forma asíncrona.
  - **dev** - Intérprete de los dispositivos especiales o de bloque del sistema de archivos.

- **nodev** - Impide la interpretación de los dispositivos especiales o de bloques del sistema de archivos.
- **suid** - Permite las operaciones de suid, y sgid bits. Se utiliza principalmente para permitir a los usuarios comunes ejecutar binarios con privilegios concedidos temporalmente con el fin de realizar una tarea específica.
- **nosuid** - Bloquea el funcionamiento de suid, y sgid bits
- **noatime** - No actualiza el inode con el tiempo de acceso al filesystem. Puede aumentar las prestaciones (véase opciones atime).
- **nodiratime** - No actualiza el inode de los directorios con el tiempo de acceso al **filesystem**. Puede aumentar las prestaciones.
- **relatime** - Actualiza en el inode solo los tiempos relativos a modificaciones o cambios de los archivos. Los tiempos de acceso vienen actualizados solo si el último acceso es anterior respecto al de la última modificación. (Similar a noatime, pero no interfiere con programas como mutt u otras aplicaciones que deben conocer si un archivo ha sido leido después de la última modificación). Puede aumentar las prestaciones (véase opciones atime).
- **discard** - Emite las órdenes TRIM para dispositivos de bloques subyacentes cuando se liberan los bloques. Recomendado para usar si el sistema de archivos se encuentra en un SSD.
- **flush** - La opción vfat permite eliminar datos con más frecuencia, de modo que los cuadros de diálogo de copia o las barras de progreso se mantenga hasta que se hayan escrito todos los datos.
- **nofail** - Monta el dispositivo cuando está presente, pero ignora su ausencia. Esto evita que se cometan errores durante el arranque para los medios extraíbles.
- defaults - Asigna las opciones de montaje predeterminadas que serán utilizadas para el sistema de archivos. Las opciones predeterminadas son: rw, suid, dev, exec, auto, nouser y async
- **dump** - Es utilizado por el programa `dump` para hacer una copia de seguridad. Dump comprueba la entrada en el archivo fstab y el número de la misma le indica si un sistema de archivos debe ser respaldado o no. La entradas posibles son 0 y 1. Si es 0, dump ignorará el sistema de archivos, mientras que si el valor es 1, dump hará una copia de seguridad.
- **pass** - Utilizado por fsck para decidir el orden en el que los sistemas de archivos serán comprobados. Las entradas posibles son 0, 1 y 2. El sistema de archivos raíz (/) debe tener la más alta prioridad (1), y todos los demás sistemas de archivos que desea comprobar deben tener un 2. La utilidad fsck no comprobará los sistemas de archivos que vengan ajustados con un valor 0

Laboratorio 9. Comprueba el archivo fstab de tu sistema y analiza cada una de las líneas. Escribe en un documento de texto que significa cada una de ellas.

Guarda la respuesta en un documento llamado Lab9-systemd.txt.

Para conocer el UUID de un dispositivo, podemos utilizar el comando

`blkid` blkid /dev/sda1

Laboratorio 10. Añade a la máquina virtual un nuevo disco.

En primer lugar es preciso particionar el disco. Para hacer esto, podemos usar la herramienta `fdisk` del siguiente modo:

fdisk /dev/sdb

Para añadir una nueva partición, usamos la opción `n`. A partir de aquí, podemos ir definiendo los números de partición, tipo de partición (primaria o extendida/lógica), y su tamaño. Finalmente, la opción `w` guardará los cambios en el disco.

Finalmente, debemos formatear la unidad, con el comando mkfs. Por ejemplo, para formatear en xfs:

mkfs.xfs /dev/sdb1

Unidades .mount

Systemd utiliza unidades con extensión **`.mount`** para el montaje de unidades de almacenamiento. Para conocer qué unidades de este tipo hay disponibles, podemos ejecutar:

find / -name "*.mount"

Comprueba el contenido del archivo `tmp.mount`

Un archivo .mount incluye varias secciones. La sección **[Mount],** es la más importante. En ella se indica qué montar, dónde hacerlo y que tipo usar. El archivo también específica en qué target se carga la unidad.

Vamos a crear una nueva unidad de montaje. Creamos un archivo **test.mount** en la ruta **`/usr/lib/systemd/system`** con el siguiente contenido:

Description = test mount

[Mount]

What =/dev/sdb1

#What =/dev/disk/by-uuid/b819c37a-f0a9-4959-9e2b-fdde26fef111

Where = /test

Type = xfs

Options=defaults

[Install]

WantedBy = multi-user.target

Estas son las opciones:

What= a partition name, path or UUID to mount

Where= an absolute path of a directory i.e. path to a mount point. If the mount point is non-existent, it will be created

Type= file system type. In most cases mount command auto-detects the file system

Options= Mount options to use when mounting

Si deseamos cargar inmediatamente la unidad, podemos hacer lo siguiente:

 # systemctl daemon-reload
#systemctl start test.mount
#systemctl stop test.mount
 # systemctl status test.mount

También podemos activar permanentemente la unidad para que se cargue durante el arranque mediante **systemd enable**.

También la podemos cargar a través del UUID, con el comando **blkid** y añadiendo la ruta persistente en almacenamiento:

by-uuid

UUID es un mecanismo para dotar a cada sistema de archivos de un identificador único. Estos identificadores son generados por las utilidades del sistema de archivos (por ejemplo, `mkfs.*`) cuando la partición se formatea y se ha diseñado de manera que evite conflictos. Todos los sistemas de archivos de GNU/Linux (incluyendo swap y las cabeceras LUKS de dispositivos cifrados en bruto) soportan UUID. Los sistemas de archivos FAT y NTFS (*fat* y etiqueta *windows* del ejemplo de abajo) no son compatibles con UUID, pero todavía se enumeran en `/dev/disk/by-uuid` con un UID más corto (identificador único):

```
$ ls -l /dev/disk/by-uuid/
```

```
total 0
lrwxrwxrwx 1 root root 10 May 27 23:31 0a3407de-014b-458b-b5c1-
848e92a327a3 -> ../../sda2
lrwxrwxrwx 1 root root 10 May 27 23:31 b411dc99-f0a0-4c87-9e05-
184977be8539 -> ../../sda3
lrwxrwxrwx 1 root root 10 May 27 23:31 CBB6-24F2 -> ../../sda1
```

```
lrwxrwxrwx 1 root root 10 May 27 23:31 f9fe0b69-a280-415d-a03a-
a32752370dee -> ../../sda4
```

La ventaja de usar el método UUID es que es mucho menos probable que se produzcan colisiones de nombres que con las etiquetas. Además, se genera automáticamente en la creación del sistema de archivos. Será, por ejemplo, el único indicador, incluso si el dispositivo está conectado a otro sistema (el cual, tal vez, puede tener un dispositivo con la misma etiqueta).

La desventaja es que UUID hace largas líneas de código difícil de leer y rompe el formato en muchos archivos de configuración (por ejemplo **fstab** o crypttab). También, cada vez que una partición se redimensiona o reformatea, se genera un nuevo UUID y las configuraciones tienen que volver a ajustarse (manualmente).

Importante:

Las unidades de montaje deben tener el nombre de los directorios de punto de montaje que controlan. Ejemplo: el punto de montaje **/mnt/backups** debe estar configurado en un archivo unit **mnt-backups.mount**

Ejemplo:

```
#vi /etc/systemd/system/mnt-backups.mount

[Unit]
Description=Mount System Backups Directory

[Mount]
What=/dev/disk/by-uuid/86fef3b2-bdc9-47fa-bbb1-4e528a89d222
Where=/mnt/backups
Type=ext4
Options=defaults
[Install]
WantedBy=multi-user.target
```

Reiniciamos systemd daemon iniciamos la unit

```
# systemctl daemon-reload
# systemctl start mnt-backups.mount
# systemctl enable mnt-backups.mount
```

 # systemctl status mnt-backups.mount

```
● mnt-backups.mount - Mount System Backups Directory
   Loaded: loaded (/etc/systemd/system/mnt-backups.mount; enabled;
vendor preset: disabled)
   Active: active (mounted) since Mon 2015-08-31 08:09:15 EAT; 2 days
ago
    Where: /mnt/backups
     What: /dev/sdc
```

```
  Process: 744 ExecMount=/bin/mount /dev/disk/by-uuid/86fef3b2-bdc9-
47fa-bbb1-4e528a89d222 /mnt/backups -n -t ext4 -o defaults
(code=exited, status=0/SUCCESS)

Aug 31 08:09:15 localhost systemd[1]: Mounting Mount System Backups
Directory...
Aug 31 08:09:15 localhost systemd[1]: Mounted Mount System Backups
Directory.
```

Laboratorio 11. Añade a la máquina virtual dos nuevos discos.

1. Tendremos que crear la unit del ejemplo **test.mount**
2. Tendremos que crear una nueva unit tipo .mount para nuestro nuevo disco, y que este montado en **/mnt/backups.**

Tendremos que comprobar que se inicia y se detiene correctamente a través de **systemd**, también se tendrá que montar al inicio del servidor.

Solución de problemas

Investigar errores de systemd

Como ejemplo, vamos a investigar un error con el servicio **systemd-modules-load**:

1. Vamos a determinar los servicios de *systemd* que fallan al inicio:

$ **systemctl --state=failed**

systemd-modules-load.service loaded **failed failed** Load Kernel Modules

2. Encontramos un problema con el servicio `systemd-modules-load`. Indaguemos un poco más:

$ systemctl status systemd-modules-load

systemd-modules-load.service - Load Kernel Modules

Loaded: loaded (/usr/lib/systemd/system/systemd-modules-load.service; static)

Active: **failed** (Result: exit-code) since So 2013-08-25 11:48:13 CEST; 32s ago

Docs: man:systemd-modules-load.service(8).

man:modules-load.d(5)

Process: **15630** ExecStart=/usr/lib/systemd/systemd-modules-load (**code=exited, status=1/FAILURE**)

3. Ahora tenemos el identificador del proceso (PID) para investigar este error en profundidad. Escribimos la siguiente orden con el `Process ID` (en este caso: 15630):

$ journalctl -b _PID=15630

-- Logs begin at Sa 2013-05-25 10:31:12 CEST, end at So 2013-08-25 11:51:17 CEST. --

Aug 25 11:48:13 mypc systemd-modules-load[15630]: **Failed to find module 'blacklist usblp'**

Aug 25 11:48:13 mypc systemd-modules-load[15630]: **Failed to find module 'install usblp /bin/false'**

4. Vemos que algunos de los ajustes del módulo del kernel tienen valores erróneos. Por lo tanto, echemos un vistazo a estos valores en `/etc/modules-load.d/`:

$ ls -Al /etc/modules-load.d/

...

-rw-r--r-- 1 root root 79 1. Dez 2012 blacklist.conf

-rw-r--r-- 1 root root 1 2. Mär 14:30 encrypt.conf

-rw-r--r-- 1 root root 3 5. Dez 2012 printing.conf

-rw-r--r-- 1 root root 6 14. Jul 11:01 realtek.conf

-rw-r--r-- 1 root root 65 2. Jun 23:01 virtualbox.conf

...

5. El mensaje del error `Failed to find module 'blacklist usblp'` puede estar relacionado con un mal ajuste de `blacklist.conf`. Podemos desactivarlo insertando un signo # delante de cada opción que hemos descubierto que falla por medio del paso 3:

/etc/modules-load.d/blacklist.conf

blacklist usblp

install usblp /bin/false

6. Ahora, intente iniciar `systemd-modules-load`:

$ systemctl start systemd-modules-load.service

Si ha tenido éxito, no debe mostrarse ningún prompt. Si ve algún error, volveremos al paso 3 y utilizaremos el nuevo PID para solucionar los errores que aparecen en la izquierda.

Si todo está bien, se puede verificar que el servicio se ha iniciado satisfactoriamente con:

$ systemctl status systemd-modules-load

systemd-modules-load.service - Load Kernel Modules

Loaded: **loaded** (/usr/lib/systemd/system/systemd-modules-load.service; static)

Active: **active (exited)** since So 2013-08-25 12:22:31 CEST; 34s ago

Docs: man:systemd-modules-load.service(8)

man:modules-load.d(5)

Process: 19005 ExecStart=/usr/lib/systemd/systemd-modules-load (code=exited, status=0/SUCCESS)

Aug 25 12:22:31 mypc systemd[1]: **Started Load Kernel Modules**.

Unit Management

Hasta ahora, hemos estado trabajando con los servicios y la visualización de información sobre los archivos de la unidad y la unidad que `systemd` conoce. Sin embargo, podemos encontrar información más específica acerca de las unidades que utilizan algunos comandos adicionales.

Viendo un archivo de Unidad

Para visualizar el archivo de unidad que `systemd`ha cargado en su sistema, puede utilizar el `cat` comando (esto se añadió en `systemd`la **versión 209**). Por ejemplo, para ver el archivo de la unidad del `atd` demonio de planificación, podríamos escribir:

```
systemctl cat atd.service
[Unit]
Description=ATD daemon

[Service]
Type=forking
ExecStart=/usr/bin/atd

[Install]
WantedBy=multi-user.target
```

Viendo dependencias

Para ver el árbol de dependencias de una unidad, se puede utilizar el comando `list-dependencies`:

```
systemctl list-dependencies sshd.service
```

Esto mostrará una jerarquía de las dependencias que deben ser tratadas con el fin de iniciar la unidad en cuestión.

```
sshd.service
├─system.slice
└─basic.target
  ├─microcode.service
  ├─rhel-autorelabel-mark.service
  ├─rhel-autorelabel.service
  ├─rhel-configure.service
  ├─rhel-dmesg.service
  ├─rhel-loadmodules.service
  ├─paths.target
  ├─slices.target
```

Comprobación de las propiedades de la unidad

Para ver las propiedades de bajo nivel de una unidad, se puede utilizar el comando `show`. Esto mostrará una lista de propiedades que se establecen para la unidad especificada usando un formato `key=value`:

```
systemctl show sshd.service

Id=sshd.service
Names=sshd.service
Requires=basic.target
Wants=system.slice
WantedBy=multi-user.target
Conflicts=shutdown.target
Before=shutdown.target multi-user.target
After=syslog.target network.target auditd.service systemd-
journald.socket basic.target system.slice
Description=OpenSSH server daemon
```

Si queremos mostrar una sola propiedad, se puede pasar la opcion `-p` con el nombre de la propiedad.

Por ejemplo, para ver los conflictos que la unidad `sshd.service` tiene, puede escribir:

```
systemctl show sshd.service -p Conflicts

Conflicts=shutdown.target
```

Editing Unit Files

A partir de la versión de **systemd 218** podremos editar y modificar archivos de units, para nuestras necesidades:

```
systemctl edit httpd.service
```

Este comando crea un archivo en blanco que puede ser utilizado para anular o añadir directivas a la definición unidad. Un directorio se creara dentro del /**etc/systemd/system** que contiene el nombre de la unidad con **.d**

Por ejemplo, para el httpd.service, creara un directorio llamado **httpd.service.d**.

Dentro de este directorio, se creara una archivo override.conf. Cuando se carga la unidad, systemd , en memoria, cargara el archivo override.conf con el archivo de la unidad completa. Las directrices del archivo override.conf tendrán prioridad sobre las que se encuentran en el archivo de unidad original.

Podremos editar el archivo de la unit en lugar de la creación de un archivo específico con el comando:

systemctl edit --full httpd.service

Esto cargará el archivo de la unidad en nuestro editor, donde puede ser modificado. Cuando se sale del editor, el archivo modificado se creara en /**etc/systemd/system**, que tendrá prioridad sobre la unidad de la definición del sistema **/lib/systemd/system.**

Para eliminar cualquier modificación que hemos creado, elimine de la unidad de .d directorio de configuración o el archivo modificado del servicio /etc/systemd/system.

Por ejemplo, para eliminar el directorio, podríamos escribir:

rm -r /etc/systemd/system/nginx.service.d

Para eliminar un archivo completo de la unidad modificada, tendríamos que escribir:

rm /etc/systemd/system/nginx.service

Después de eliminar el archivo o directorio, se debe volver a cargar el procedo de systemd, de modo que ya no intente hacer referencia a estos archivos

systemctl daemon-reload

Laboratorio 12 Instalación servidor Tomcat y gestionado a través de Systemd.

En este laboratorio instalaremos en nuestro servidor de Linux, un Tomcat, el cual tendremos que gestionar el servicio a través de Systemd

Configuraremos el servicio de systemd para tomcat, para que se inicie con el servidor de Linux, nuestro servidor de Tomcat.

Tendremos que ajustar nuestro script de inicio a las variables de nuestra instalación del servidor Tomcat, y dar los permisos necesarios a los archivos .sh del directorio bin.

vi /etc/systemd/system/tomcat.service

[Unit]

Description=Apache Tomcat Web Application Container

After=network.target

[Service]

Type=forking

Environment=JAVA_HOME=/usr/lib/jvm/java-1.7.0-openjdk-1.7.0.75-2.5.4.2.el7_0.x86_64/jre

Environment=CATALINA_PID= /tomcat-8/temp/tomcat.pid

Environment=CATALINA_HOME=/tomcat-8

Environment=CATALINA_BASE=/tomcat-8

Environment='CATALINA_OPTS=-Xms512M -Xmx1024M -server -XX:+UseParallelGC'

ExecStart= /tomcat-8/bin/startup.sh

ExecStop= /tomcat-8/bin/shutdown.sh

User=root

Group=root

UMask=0007

RestartSec=10

Restart=always

[Install]

WantedBy=multi-user.target

```
#systemctl daemon-reload
#systemctl start tomcat
#systemctl status tomcat
#systemctl enable tomcat
```

Laboratorio 13 Modificación de script de Apache, a través de systemd.

En este laboratorio necesitamos personalizar la .unit de apache, para que cuando se inicie el servicio de Apache, se inicie automáticamente el servicio de Tomcat y cuando paremos el servicio de Apache se pare el servicio de Tomcat, todo a través de Systemd.

Para realizar este laboratorio podemos apoyarnos en el documento:

Red_Hat_Enterprise_Linux-7-System_Administrators_Guide-en-US.pdf

Pag: 118, 128

Importante:

Podemos copiar el archivo de la antigua unidad desde **/usr/lib/systemd/system**/ a /**etc/systemd/system**/ y realizar los cambios allí.

Un archivo de unidad ubicado en /**etc/systemd/system/** siempre tiene preferencia sobre la misma unidad localizada en /usr/lib/systemd/system/.

Debemos tener en cuenta que cuando la unidad original localizada en /usr/lib/ ha cambiado debido a una actualización del paquete que lo suministra, estos cambios no se aplicarán automáticamente al archivo de unidad personalizada ubicado en /etc/. De este modo, tendremos que volver a activar manualmente la unidad con la orden:

systemctl reenable *unidad*.

Por consiguiente, se recomienda utilizar el método *.conf descrito anteriormente

Podemos utilizar la orden **systemd-delta** para ver qué archivos de unidad han sido invalidados y cuáles han cambiado.